

Application of Euclidean Distance in K-Nearest Neighbor Regression

Devi Prasad M

August 2026

Euclidean Distance – An Application: KNN Regression

A 4D Example: Predicting House Prices

Consider a 4-dimensional feature space where we want to predict the sale price of a house based on its characteristics. Our features are:

- $x^{(1)}$: Living Area (in 1,000 sq ft) – *Continuous*
- $x^{(2)}$: Distance to City Center (kms) – *Continuous*
- $x^{(3)}$: Age of Property (years) – *Integer*
- $x^{(4)}$: Number of Bedrooms – *Integer*

Our target y is the **Sale Price (in INR Crores)**.

Suppose a new house comes on the market. Our query point is a 2,000 sq ft home, 1.0 km from the center, 10 years old, with 3 bedrooms:

$$\mathbf{x}_q = (2.0, 1.0, 10, 3)$$

Suppose our historical training dataset contains the following 12 recent sales:

Property	$x^{(1)}$ (Area)	$x^{(2)}$ (Dist)	$x^{(3)}$ (Age)	$x^{(4)}$ (Beds)	y_i (Price)
$\mathbf{x}_1$	1.5	1.5	12	3	3.5
$\mathbf{x}_2$	2.2	0.5	8	4	5.2
$\mathbf{x}_3$	2.0	1.0	9	3	4.8
$\mathbf{x}_4$	1.8	1.2	10	3	4.6
$\mathbf{x}_5$	2.5	2.0	5	4	6.0
$\mathbf{x}_6$	1.2	3.0	20	2	2.5
$\mathbf{x}_7$	2.1	0.8	11	3	4.9
$\mathbf{x}_8$	3.0	5.0	2	5	7.0
$\mathbf{x}_9$	1.9	1.5	10	2	4.1
$\mathbf{x}_{10}$	2.0	0.5	15	3	4.3
$\mathbf{x}_{11}$	2.3	1.1	10	4	5.4
$\mathbf{x}_{12}$	1.7	1.0	8	3	4.4

We choose $K = 3$.

Step 1: Measure the Distance to Every Point

The 4-dimensional Euclidean distance to any training point $\mathbf{x}_i$ is:

$$\text{dist}(\mathbf{x}_q, \mathbf{x}_i) = \sqrt{(2.0 - x_i^{(1)})^2 + (1.0 - x_i^{(2)})^2 + (10 - x_i^{(3)})^2 + (3 - x_i^{(4)})^2}$$

We calculate this geometric distance to *every* one of the 12 properties:

Property	y_i (Price)	Distance	Property	y_i (Price)	Distance
$\mathbf{x}_1$	3.5	2.12	$\mathbf{x}_7$	4.9	1.02
$\mathbf{x}_2$	5.2	2.30	$\mathbf{x}_8$	7.0	9.22
$\mathbf{x}_3$	4.8	1.00	$\mathbf{x}_9$	4.1	1.12
$\mathbf{x}_4$	4.6	0.28	$\mathbf{x}_{10}$	4.3	5.02
$\mathbf{x}_5$	6.0	5.22	$\mathbf{x}_{11}$	5.4	1.05
$\mathbf{x}_6$	2.5	10.28	$\mathbf{x}_{12}$	4.4	2.02

Measure: Every training point is compared with the query.

Step 2: Select the K Nearest Neighbors

Since $K = 3$, we select the three properties with the smallest Euclidean distances:

Neighbor	Target y_i	$\text{dist}(\mathbf{x}_q, \mathbf{x}_i)$
$\mathbf{x}_4 = (1.8, 1.2, 10, 3)$	4.6	0.28
$\mathbf{x}_3 = (2.0, 1.0, 9, 3)$	4.8	1.00
$\mathbf{x}_7 = (2.1, 0.8, 11, 3)$	4.9	1.02

Therefore, our nearest neighborhood is:

$$N_3(\mathbf{x}_q) = \{(\mathbf{x}_4, 4.6), (\mathbf{x}_3, 4.8), (\mathbf{x}_7, 4.9)\}.$$

Step 3: Predict

The KNN regression prediction is the arithmetic mean of the target prices of these three most similar houses:

$$\hat{y}(\mathbf{x}_q) = \frac{4.6 + 4.8 + 4.9}{3} = \frac{14.3}{3} \approx 4.77.$$

The KNN estimate for the unknown market value of our query house is:

Estimated Price $\approx$ INR 4.77 Crores

A Critical Flaw: The Need for Feature Scaling

While the example above perfectly demonstrates the mechanics of KNN, there is a hidden mathematical flaw in applying Euclidean distance directly to raw, unscaled real-world data.

Consider the differences in scale among our features:

- A difference of 5 years in **Age** contributes $(5)^2 = 25$ to the distance sum.
- A difference of 2 kms in **Distance** contributes only $(2)^2 = 4$ to the distance sum.

Even if distance to the city center has a more massive impact on actual property value, the **Age** feature mathematically dominates the Euclidean distance simply because its numerical values are larger. In effect, we are comparing apples to oranges.

The Solution: Standardization

To prevent variables with larger scales from biasing the distance metric, distance-based algorithms require data preprocessing. We must normalize or standardize the features so they contribute equally to the geometry.

A standard approach is *standardization* or *Z-score normalization*, which transforms every feature to have a mean of 0 and a standard deviation of 1:

$$x_{\text{scaled}}^{(j)} = \frac{x^{(j)} - \mu_j}{\sigma_j}$$

where μ_j is the mean and σ_j is the standard deviation of feature j .